

WEEK-9

Create Database:

CREATE DATABASE kluh;

The screenshot shows the MySQL Workbench interface. The left sidebar contains the 'MANAGEMENT' tree with options like Server Status, Client Connections, and Schemas. The main editor window displays a SQL query with three lines: `use kluh;`, `select * from products;`, and `show databases;`. Below the query editor, the 'Result Grid' shows the output of the queries. The first result shows the list of databases, including 'blood_bank', 'blood_bank_db', 'bookflow_db', 'bookflow_db1', 'db2', and 'dbms_lab'. The second result shows the output of the 'select * from products' query, which returned 4 rows. The third result shows the output of the 'show databases' query, which returned 18 rows.

Database
blood_bank
blood_bank_db
bookflow_db
bookflow_db1
db2
dbms_lab

#	Time	Action	Message
1	09:06:12	use kluh	0 row(s) affected
2	09:06:26	select * from products LIMIT 0, 1000	4 row(s) returned
3	09:29:13	show databases	18 row(s) returned

Create table

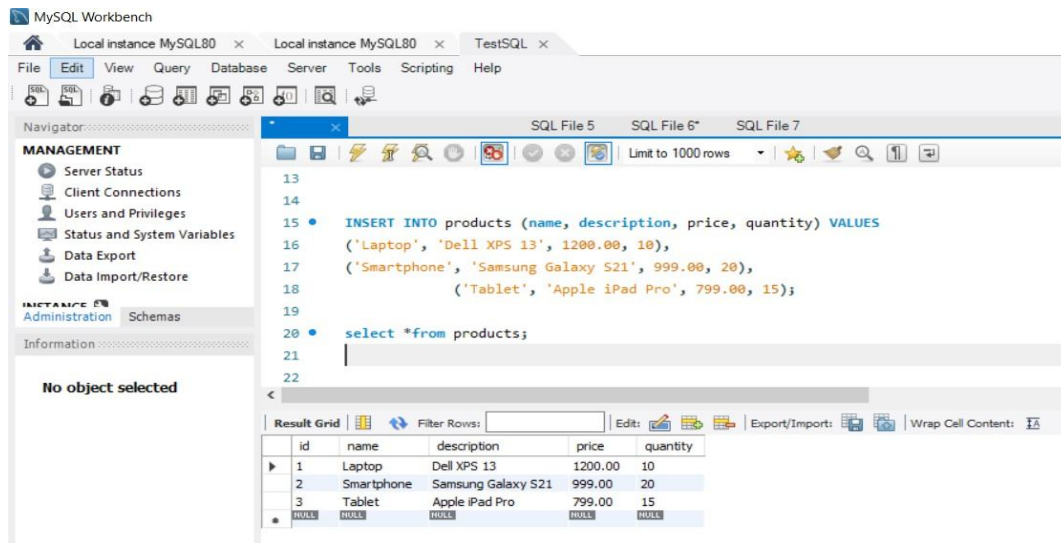
The screenshot shows the MySQL Workbench interface. The left sidebar contains the 'MANAGEMENT' tree. The main editor window displays a SQL query to create a table named 'products'. The query is as follows:

```
CREATE TABLE products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  description TEXT,  
  price DECIMAL(10,2) NOT NULL,  
  quantity INT NOT NULL  
);  
select * from products;
```

Below the query editor, the 'Result Grid' shows the output of the 'select * from products' query, which returned 4 rows. The columns are 'id', 'name', 'description', 'price', and 'quantity'.

id	name	description	price	quantity
1	...	...	...	...
2	...	...	...	...
3	...	...	...	...
4	...	...	...	...

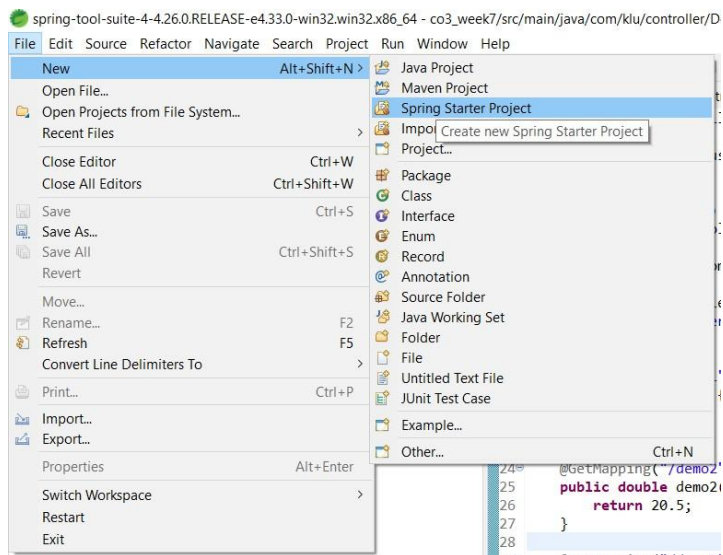
Insert values



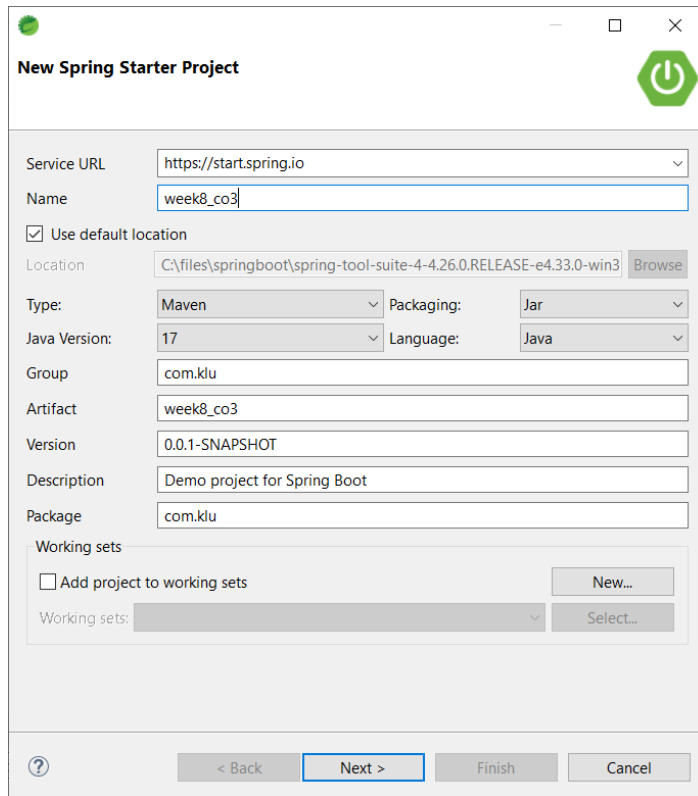
PROJECT CREATION

→BACKEND

STEP-1:



STEP-2:



The image shows a 'New Spring Starter Project' dialog box. It contains several input fields and dropdown menus for configuring a new project. The 'Service URL' is set to 'https://start.spring.io'. The 'Name' is 'week8_co3'. The 'Location' is 'C:\files\springboot\spring-tool-suite-4-4.26.0.RELEASE-e4.33.0-win3'. The 'Type' is 'Maven', 'Packaging' is 'Jar', 'Java Version' is '17', and 'Language' is 'Java'. The 'Group' is 'com.klu', 'Artifact' is 'week8_co3', 'Version' is '0.0.1-SNAPSHOT', 'Description' is 'Demo project for Spring Boot', and 'Package' is 'com.klu'. There is a 'Working sets' section with a checkbox 'Add project to working sets' and a 'New...' button. Below it is a 'Working sets:' dropdown and a 'Select...' button. At the bottom are buttons for '< Back', 'Next >', 'Finish', and 'Cancel'.

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

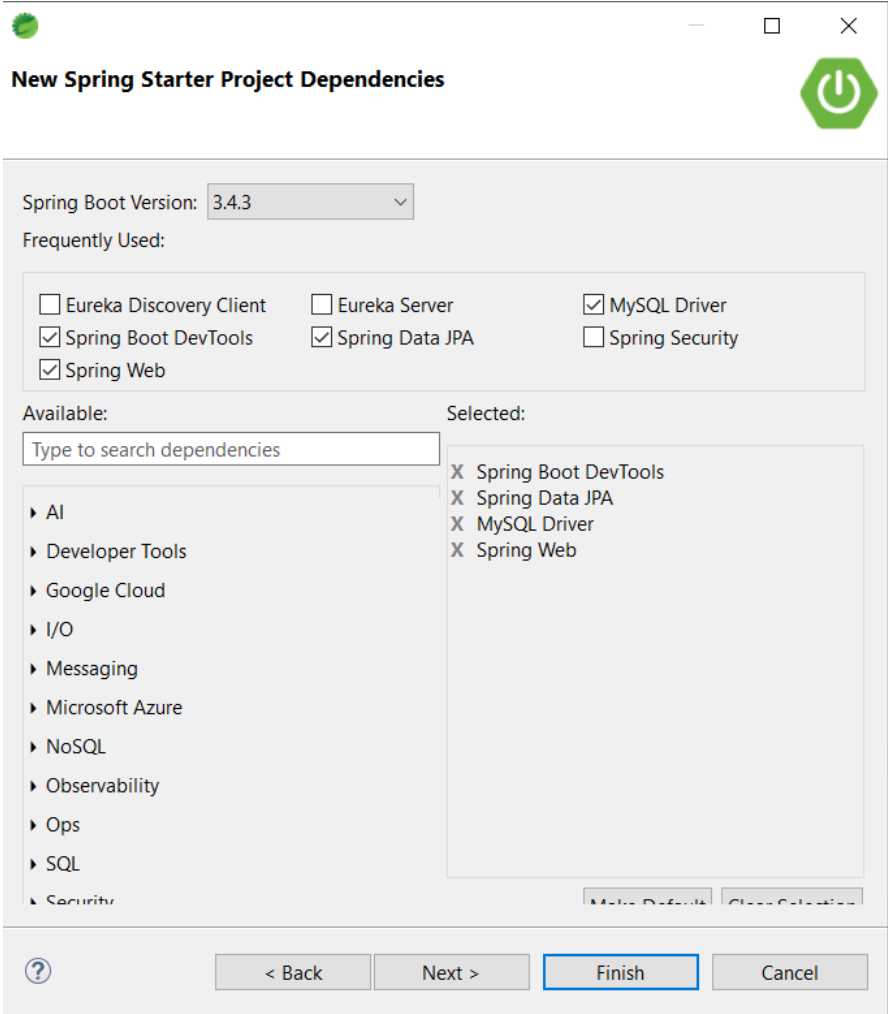
Package:

Working sets

☐ Add project to working sets

Working sets:

STEP-3:



The dialog box is titled "New Spring Starter Project Dependencies" and features a green power button icon in the top right corner. It contains a "Spring Boot Version" dropdown menu set to "3.4.3". Below this is a "Frequently Used" section with checkboxes for "Eureka Discovery Client", "Eureka Server", "MySQL Driver", "Spring Boot DevTools", "Spring Data JPA", "Spring Security", and "Spring Web". The "Available:" section on the left lists categories like AI, Developer Tools, Google Cloud, I/O, Messaging, Microsoft Azure, NoSQL, Observability, Ops, SQL, and Security. The "Selected:" section on the right shows "Spring Boot DevTools", "Spring Data JPA", "MySQL Driver", and "Spring Web" with an 'X' next to each. At the bottom, there are buttons for "< Back", "Next >", "Finish", and "Cancel".

New Spring Starter Project Dependencies

Spring Boot Version: 3.4.3

Frequently Used:

☐ Eureka Discovery Client ☐ Eureka Server ☒ MySQL Driver
☒ Spring Boot DevTools ☒ Spring Data JPA ☐ Spring Security
☒ Spring Web

Available: Selected:

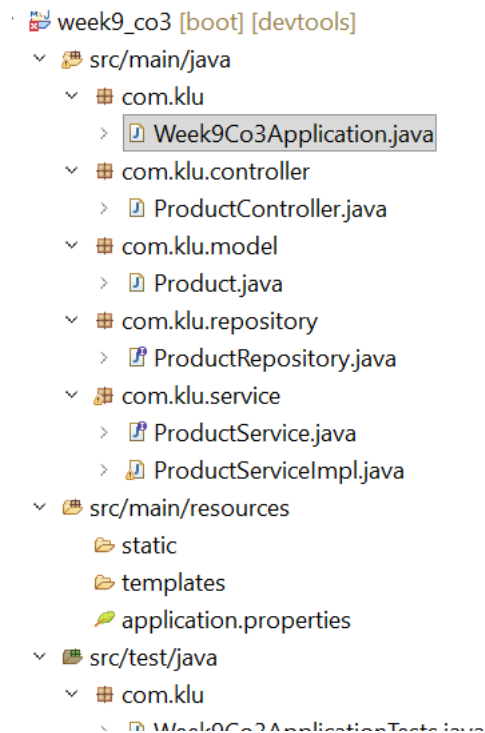
Type to search dependencies

- ▶ AI
- ▶ Developer Tools
- ▶ Google Cloud
- ▶ I/O
- ▶ Messaging
- ▶ Microsoft Azure
- ▶ NoSQL
- ▶ Observability
- ▶ Ops
- ▶ SQL
- ▶ Security

- X Spring Boot DevTools
- X Spring Data JPA
- X MySQL Driver
- X Spring Web

Make Default Clear Selection

? < Back Next > Finish Cancel



PROGRAMS

ProductController.java

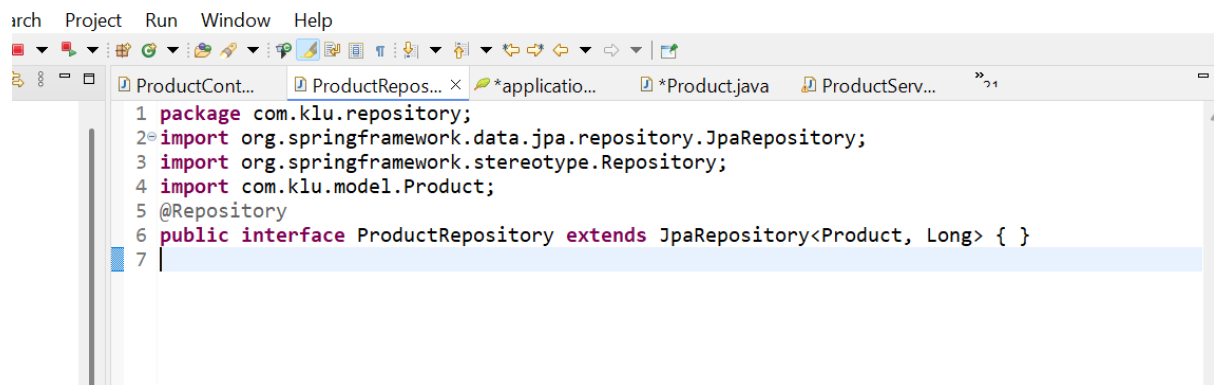
```
ProductCont... × *applicatio... student-res... week8_co3/po... ProductServ... ?1
1 package com.klu.controller;
2 import com.klu.model.Product;
3 import com.klu.service.ProductService;
4 import org.springframework.web.bind.annotation.*;
5 import java.util.List;
6 @RestController
7 @RequestMapping("/api/products")
8 @CrossOrigin(origins = "http://localhost:5173")
9 public class ProductController {
10 private final ProductService productService;
11 public ProductController(ProductService productService) {
12 this.productService = productService;
13 }
14 @GetMapping
15 public List<Product> getAllProducts() {
16 return productService.getAllProducts();
17 }
18 @GetMapping("/{id}")
19 public Product getProductById(@PathVariable Long id) {
20 return productService.getProductById(id);
21 }
22 @PostMapping
23 public Product addProduct(@RequestBody Product product) {
24 return productService.saveProduct(product);
25 }
26 @PutMapping("/{id}")
27 public Product updateProduct(@PathVariable Long id, @RequestBody Product product) {
28 return productService.updateProduct(id, product);
29 }
30 @DeleteMapping("/{id}")
31 public void deleteProduct(@PathVariable Long id) {
32 productService.deleteProduct(id);
33 }
34 }
```

Product.java

```
ProductCont... *applicatio... *Product.java × student-res... week8_co3/po... Pro
1 package com.klu.model;
2 import jakarta.persistence.*;
3 @Entity
4 @Table(name = "products")
5 public class Product {
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9     private String name;
10    private String description;
11    private double price;
12    private int quantity;
13    public Long getId() {
14        return id;
15    }
16    public void setId(Long id) {
17        this.id = id;
18    }
19    public String getName() {
20        return name;
21    }
22    public void setName(String name) {
23        this.name = name;
24    }
25    public String getDescription() {
26        return description;
27    }
28    public void setDescription(String description) {
29        this.description = description;
30    }
31    public double getPrice() {
32        return price;
33    }
34    public void setPrice(double price) {
35        this.price = price;
36    }
```

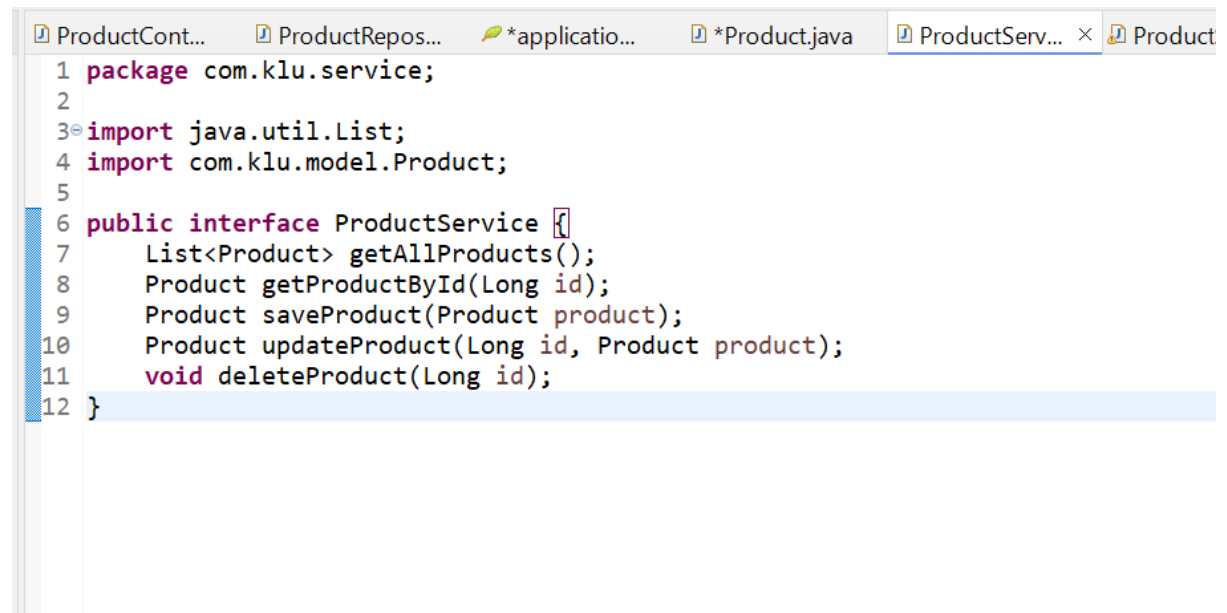
```
28    public void setDescription(String description) {
29        this.description = description;
30    }
31    public double getPrice() {
32        return price;
33    }
34    public void setPrice(double price) {
35        this.price = price;
36    }
37    public int getQuantity() {
38        return quantity;
39    }
40    public void setQuantity(int quantity) {
41        this.quantity = quantity;
42    }
43    }
44
```

ProductRepository.java



```
1 package com.klu.repository;
2 import org.springframework.data.jpa.repository.JpaRepository;
3 import org.springframework.stereotype.Repository;
4 import com.klu.model.Product;
5 @Repository
6 public interface ProductRepository extends JpaRepository<Product, Long> { }
7
```

ProductService.java(interface)



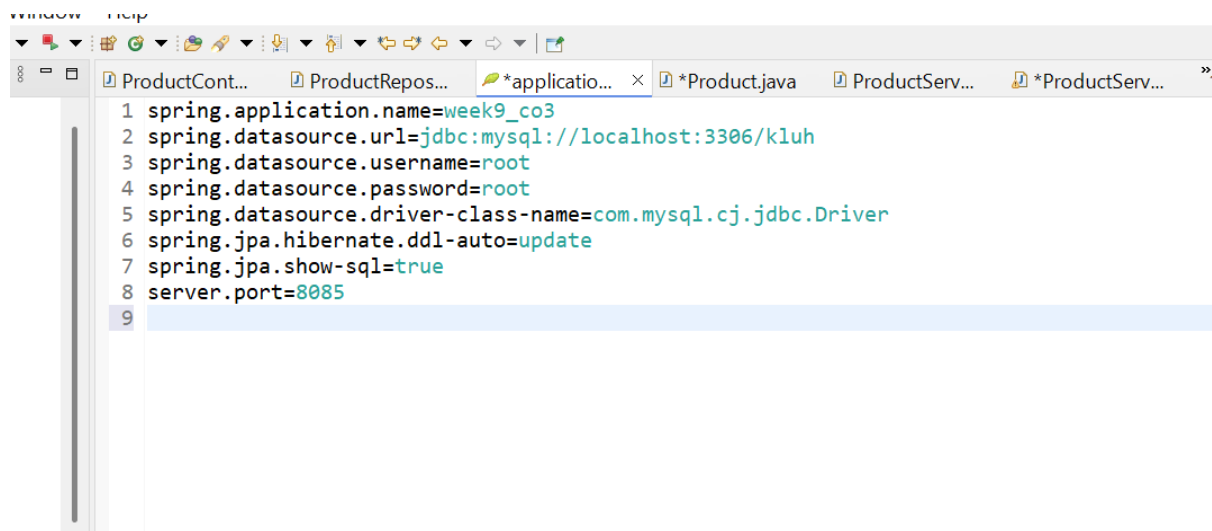
```
1 package com.klu.service;
2
3 import java.util.List;
4 import com.klu.model.Product;
5
6 public interface ProductService {
7     List<Product> getAllProducts();
8     Product getProductById(Long id);
9     Product saveProduct(Product product);
10    Product updateProduct(Long id, Product product);
11    void deleteProduct(Long id);
12 }
```

ProductServiceImpl.java

```
ProductCont... ProductRepos... *applicatio... *Product.java ProductServ... *
1 package com.klu.service;
2 import java.util.List;
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Service;
5 import com.klu.model.Product;
6 import com.klu.repository.ProductRepository;
7 @Service
8 public class ProductServiceImpl implements ProductService {
9     private final ProductRepository productRepository;
10    @Autowired
11    public ProductServiceImpl(ProductRepository productRepository) {
12        this.productRepository = productRepository;
13    }
14    @Override
15    public List<Product> getAllProducts() {
16        return productRepository.findAll();
17    }
18    @Override
19    public Product getProductById(Long id) {
20        return productRepository.findById(id).orElse(null);
21    }
22    @Override
23    public Product saveProduct(Product product) {
24        return productRepository.save(product);
25    }
26    @Override
27    public Product updateProduct(Long id, Product product) {
28        if (productRepository.existsById(id)) {
29            product.setId(id);
30            return productRepository.save(product);
31        }
32        return null;
33    }
34    @Override
35    public void deleteProduct(Long id) {
36        productRepository.deleteById(id);
37    }
38 }
```

```
20        return productRepository.findById(id).orElse(null);
21    }
22    @Override
23    public Product saveProduct(Product product) {
24        return productRepository.save(product);
25    }
26    @Override
27    public Product updateProduct(Long id, Product product) {
28        if (productRepository.existsById(id)) {
29            product.setId(id);
30            return productRepository.save(product);
31        }
32        return null;
33    }
34    @Override
35    public void deleteProduct(Long id) {
36        productRepository.deleteById(id);
37    }
38 }
```

applications.properties

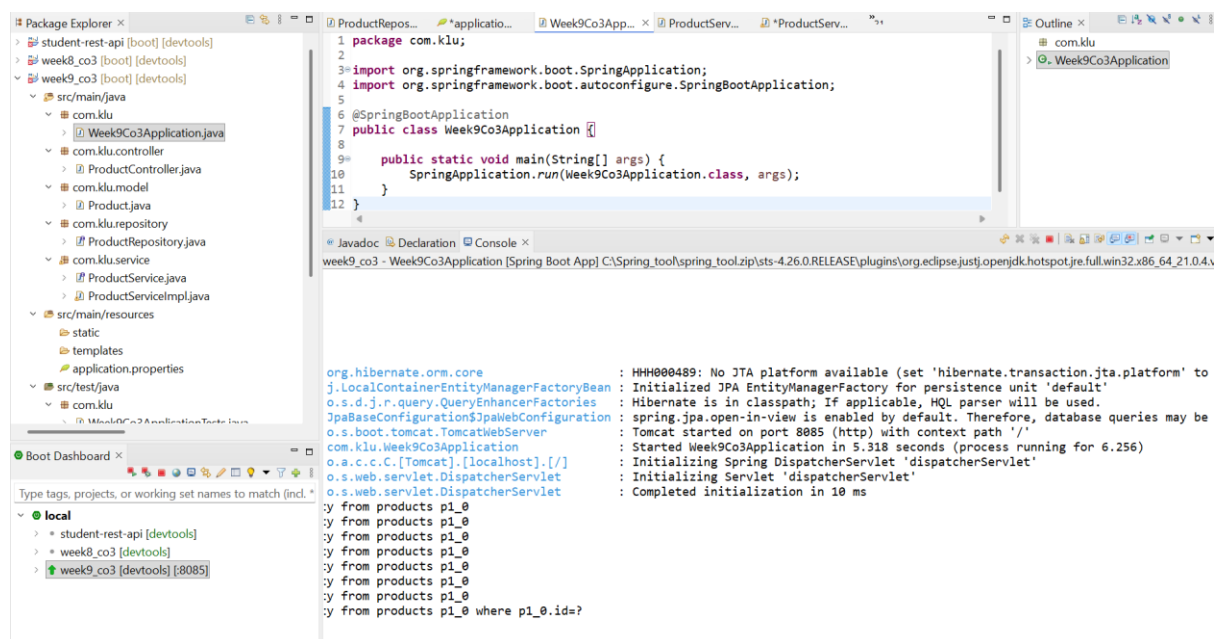


The screenshot shows an IDE window with the file 'applications.properties' open. The file contains the following configuration:

```
1 spring.application.name=week9_co3
2 spring.datasource.url=jdbc:mysql://localhost:3306/kluh
3 spring.datasource.username=root
4 spring.datasource.password=root
5 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
6 spring.jpa.hibernate.ddl-auto=update
7 spring.jpa.show-sql=true
8 server.port=8085
9
```

Run:

com.klu → Week9Co3Application.java



The screenshot shows an IDE with the 'Week9Co3Application.java' file open. The code is as follows:

```
1 package com.klu;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class Week9Co3Application {
8
9     public static void main(String[] args) {
10         SpringApplication.run(Week9Co3Application.class, args);
11     }
12 }
```

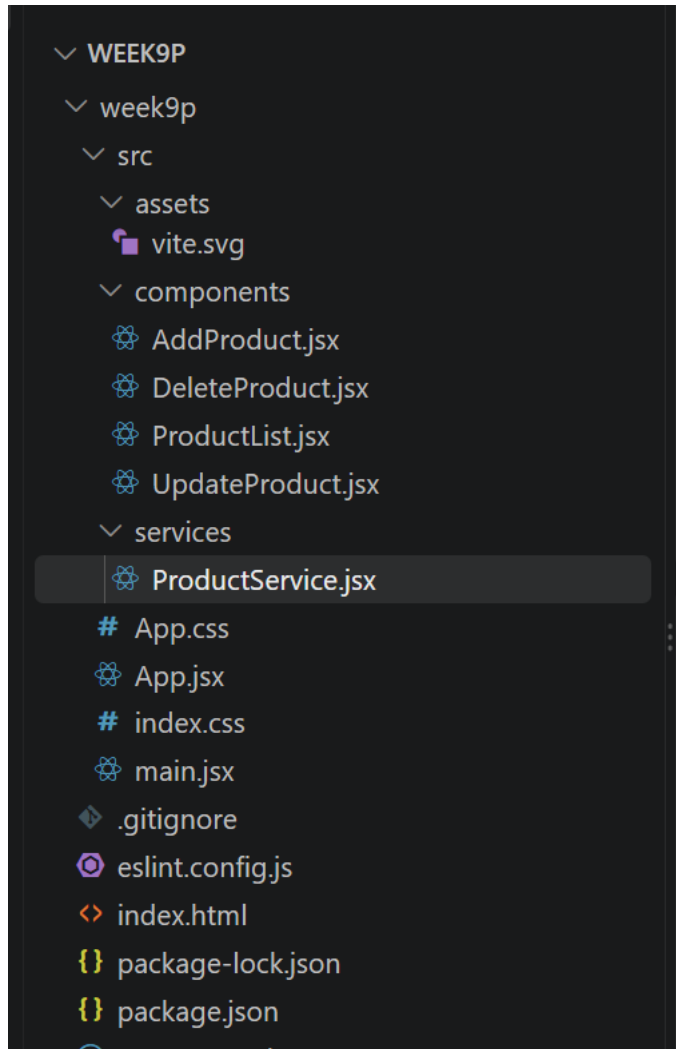
The console output shows the following logs:

```
org.hibernate.orm.core : HH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to
j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
jpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be
o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8085 (http) with context path '/'
com.klu.Week9Co3Application : Started Week9Co3Application in 5.318 seconds (process running for 6.256)
o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
o.s.web.servlet.DispatcherServlet : Completed initialization in 10 ms
y from products p1_0
y from products p1_0
y from products p1_0
y from products p1_0
y from products p1_0
y from products p1_0
y from products p1_0
y from products p1_0 where p1_0.id=?
```

➔ FRONTEND

1. Create folder(WEEK9)
2. OPEN VS code
3. Open terminal → new terminal
4. Run: npm create vite@latest week9p
5. Select react , javascript
6. All installations will be done

7. Run: npm install
8. Run: npm install axios
9. Run: npm install react-router-dom
10. under src create components folder
11. under components create .js files



AddProduct.jsx:

```
import React, { useState } from 'react';
import { addProduct } from '../services/ProductService';
const AddProduct = () => {
  const [product, setProduct] = useState({ name: "", description: "", price: "", quantity:
  "" });
  const handleChange = (e) => {
    setProduct({ ...product, [e.target.name]: e.target.value });
  };
  const handleSubmit = (e) => {
```

```

e.preventDefault();
addProduct(product).then(() => {
  alert("Product added successfully!");
});
};
return (
  <form onSubmit={handleSubmit}>
    <input type="text" name="name" placeholder="Name"
      onChange={handleChange} required />
    <input type="text" name="description" placeholder="Description"
      onChange={handleChange} />
    <input type="number" name="price" placeholder="Price"
      onChange={handleChange} required />
    <input type="number" name="quantity" placeholder="Quantity"
      onChange={handleChange} required />
    <button type="submit">Add Product</button>
  </form>
);
};
export default AddProduct;

```

DeleteProduct.jsx:

```

import React from "react";
import { useNavigate, useParams } from "react-router-dom";
import { deleteProduct } from "../services/ProductService";
const DeleteProduct = () => {
  const { id } = useParams(); // Get product ID from URL params
  const navigate = useNavigate();
  const handleDelete = async () => {
    try {
      await deleteProduct(id);
      alert("Product deleted successfully!");
      navigate("/"); // Redirect to product list
    } catch (error) {
      console.error("Error deleting product:", error);
      alert("Failed to delete product");
    }
  };
  return (
    <div>

```

```

<h2>Are you sure you want to delete this product?</h2>
<button onClick={handleDelete}>Yes, Delete</button>
<button onClick={() => navigate("/")}>Cancel</button>
</div>
);
};
export default DeleteProduct;

```

ProductList.jsx:

```

import React, { useEffect, useState } from 'react';
import { getAllProducts, deleteProduct } from '../services/ProductService';
const ProductList = () => {
  const [products, setProducts] = useState([]);
  useEffect(() => {
    getAllProducts().then(response => {
      setProducts(response.data);
    });
  }, []);
  const handleDelete = (id) => {
    deleteProduct(id).then(() => {
      setProducts(products.filter(product => product.id !== id));
    });
  };
  return (
    <div>
      <h2>Product List</h2>
      <ul>
        {products.map(product => (
          <li key={product.id}>
            {product.name} - ${product.price}
            <button onClick={() => handleDelete(product.id)}>Delete</button>
          </li>
        ))}
      </ul>
    </div>
  );
};
export default ProductList;

```

UpdateProduct.jsx

```
import React, { useState, useEffect } from "react";
import { useParams, useNavigate } from "react-router-dom";
import { updateProduct, getAllProducts } from "../services/ProductService";
const UpdateProduct = () => {
  const { id } = useParams(); // Get product ID from route params
  const navigate = useNavigate();
  const [product, setProduct] = useState({
    name: "",
    description: "",
    price: 0,
    quantity: 0,
  });
  // Fetch product details when component mounts
  useEffect(() => {
    getAllProducts().then((response) => {
      const productData = response.data.find((p) => p.id.toString() === id);
      if (productData) {
        setProduct(productData);
      }
    });
  }, [id]);
  const handleChange = (e) => {
    setProduct({ ...product, [e.target.name]: e.target.value });
  };
  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      await updateProduct(id, product);
      alert("Product updated successfully!");
      navigate("/"); // Redirect to product list
    } catch (error) {
      console.error("Error updating product:", error);
      alert("Failed to update product");
    }
  };
  return (
    <div>
      <h2>Update Product</h2>
      <form onSubmit={handleSubmit}>
```

```

    <label>Name:</label>
    <input type="text" name="name" value={product.name}
onChange={handleChange} required />
    <label>Description:</label>
    <input type="text" name="description" value={product.description}
onChange={handleChange} required />
    <label>Price:</label>
    <input type="number" name="price" value={product.price}
onChange={handleChange} required />
    <label>Quantity:</label>
    <input type="number" name="quantity" value={product.quantity}
onChange={handleChange} required />
    <button type="submit">Update Product</button>
  </form>
</div>
);
};
export default UpdateProduct;

```

12. Create service under src

13. Create file ProductService.jsx

ProductService.jsx

```

import axios from 'axios';
const API_URL = "http://localhost:8085/api/products";
export const getAllProducts = () => axios.get(API_URL);
export const addProduct = (product) => axios.post(API_URL, product);
export const updateProduct = (id, product) => axios.put(`${API_URL}/${id}`,
product);
export const deleteProduct = (id) => axios.delete(`${API_URL}/${id}`);

```

App.css

```

#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

```

```
.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffff);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafb);
}
```

```
@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}
```

```
@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}
```

```
.card {
  padding: 2em;
}
```

```
.read-the-docs {
  color: #888;
}
```

App.jsx

```
import React from "react";
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import ProductList from "../components/ProductList";
```

```

import AddProduct from "./components/AddProduct";
import UpdateProduct from "./components/UpdateProduct";
import DeleteProduct from "./components/DeleteProduct";
const App = () => {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<ProductList />} />
        <Route path="/add" element={<AddProduct />} />
        <Route path="/update/:id" element={<UpdateProduct />} />
        <Route path="/delete/:id" element={<DeleteProduct />} />
      </Routes>
    </Router>
  );
};
export default App;

```

main.jsx

```

import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

```

index.css

```

:root {
  font-family: system-ui, Avenir, Helvetica, Arial, sans-serif;
  line-height: 1.5;
  font-weight: 400;

  color-scheme: light dark;
  color: rgba(255, 255, 255, 0.87);
  background-color: #242424;

```

```
font-synthesis: none;
text-rendering: optimizeLegibility;
-webkit-font-smoothing: antialiased;
-moz-osx-font-smoothing: grayscale;
}
```

```
a {
  font-weight: 500;
  color: #646cff;
  text-decoration: inherit;
}
a:hover {
  color: #535bf2;
}
```

```
body {
  margin: 0;
  display: flex;
  place-items: center;
  min-width: 320px;
  min-height: 100vh;
}
```

```
h1 {
  font-size: 3.2em;
  line-height: 1.1;
}
```

```
button {
  border-radius: 8px;
  border: 1px solid transparent;
  padding: 0.6em 1.2em;
  font-size: 1em;
  font-weight: 500;
  font-family: inherit;
  background-color: #1a1a1a;
  cursor: pointer;
  transition: border-color 0.25s;
}
button:hover {
```

```

border-color: #646cff;
}
button:focus,
button:focus-visible {
outline: 4px auto -webkit-focus-ring-color;
}

```

```

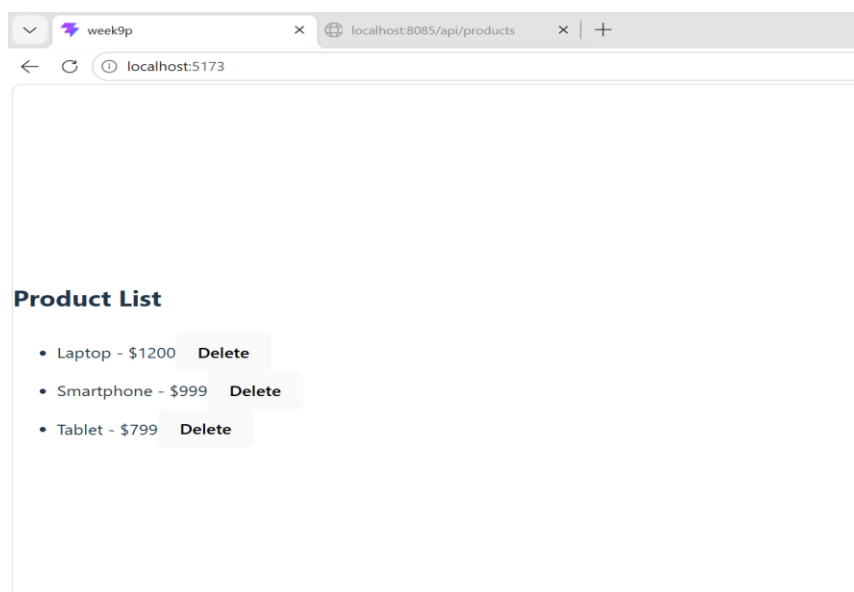
@media (prefers-color-scheme: light) {
:root {
color: #213547;
background-color: #ffffff;
}
a:hover {
color: #747bff;
}
button {
background-color: #f9f9f9;
}
}

```

14. Run the frontend :

Go to terminal
cd week9p
npm run dev

15. Link will be generated (open the link)



localhost:5173/add

localhost:5173 says
Product added successfully!

OK

mobile	vivo	13000	20	Add Product
--------	------	-------	----	-------------

localhost:5174/update/1

localhost:5174 says
Product updated successfully!

OK

Update Product

Name:LaptopDescription:Dell XPS 13Price:12000Quantity:12Update Product

week9plocalhost:8085/api/products

localhost:5174/delete/3

localhost:5174 says
Product deleted successfully!

OK

Are you sure you want to delete this product?

Yes, DeleteCancel

ily with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

GET http://localhost:8085/a

http://localhost:8085/api/products

GET http://localhost:8085/api/products

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Ram",
3   "department": "CSE",
4   "email": "ram@gmail.com"
5 }
```

Body 200 OK • 74 ms • 567 B

{ } JSON Preview Visualize

```
1 [
2   {
3     "description": "Dell XPS 13",
4     "id": 1,
5     "name": "Laptop",
6     "price": 1200.0,
7     "quantity": 10
8   },
9   {
10    "description": "Samsung Galaxy S21",
11    "id": 2,
12    "name": "Smartphone",
```

week9p localhost:8085/api/products

localhost:8085/api/products

Pretty-print

```
[{"description": "Dell XPS 13", "id": 1, "name": "Laptop", "price": 1200.0, "quantity": 10}, {"description": "Samsung Galaxy S21", "id": 2, "name": "Smartphone", "price": 999.0, "quantity": 20}, {"description": "Apple iPad Pro", "id": 3, "name": "Tablet", "price": 799.0, "quantity": 15}, {"description": "dlink", "id": 4, "name": "WifiRouter", "price": 11200.0, "quantity": 98}]
```